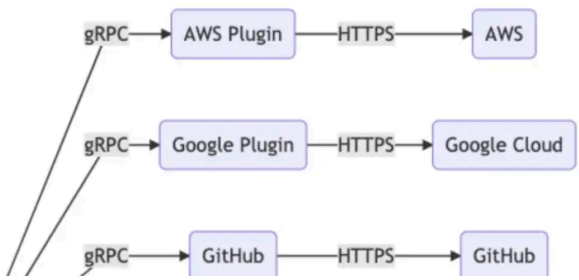
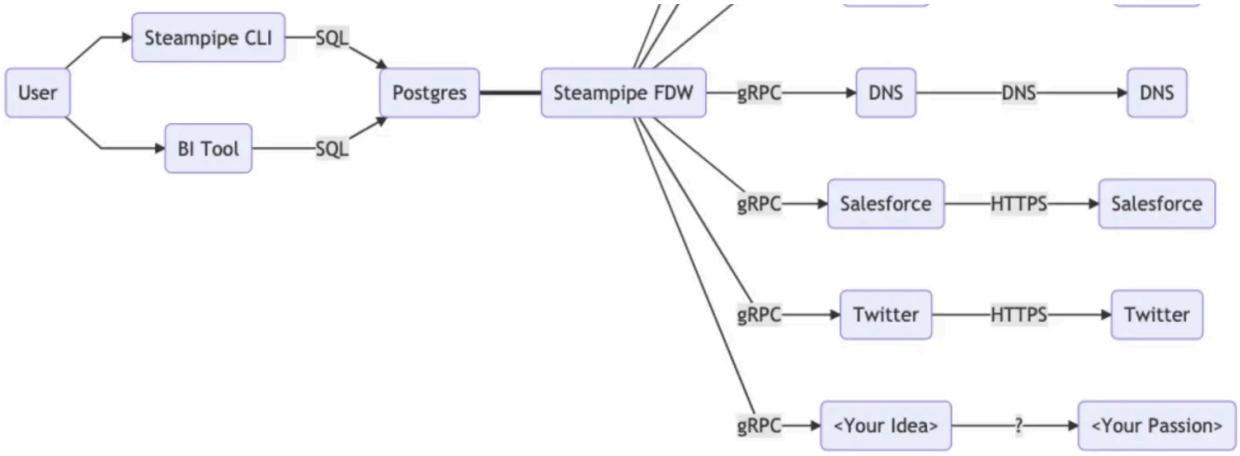
🚀 Launch Week 13: June 8th - 12th, 2026 🚀

Steampipe Architecture

Overview





Steampipe uses a Postgres Foreign Data Wrapper to present data from external systems and services as database tables. The Steampipe Foreign Data Wrapper (FDW) provides a Postgres extension that allows Postgres to connect to external data in a standardized way. The Steampipe FDW does not directly interface with external systems, but instead relies on plugins to implement the API/provider specific code and return it in a standard format via gRPC. This approach simplifies extending Steampipe as the Postgres-specific logic is encapsulated in the FDW, and API and service specific code resides only in the plugin.

Design Principles

It should "just work"

One of the goals of Steampipe since we first started envisioning it is that it should be simple to install and use - you should not need to spend hours downloading pre-requisites, fiddling with config files, setting up credentials, or pouring over documentation. We've tried very hard to bring that vision to reality, and hope that it is reflected in Steampipe as well as our plugins.

When writing plugins, attempt to make it work out-of-the box as much as possible:

- Use the vendor's CLI default credential mechanism and resolution order (if applicable). For example, we use the normal `aws` cli credentials for our `aws` plugin - `select * from aws_ec2_instance` works the same as `aws ec2 describe-instances`, using the AWS credentials file and/or standard environment variables.
- Use sane defaults that align with the vendor's cli tool, api, or UI. Configuration options should be exactly that - *optional*.
- Where possible, avoid any dependence on other 3rd party tools or libraries that are not compiled into your plugin binary.

It should feel simple, intuitive, and familiar

We chose SQL as the language for Steampipe as much for its ubiquity as its power - It was invented in 1970, and became an ANSI standard in 1986. Most developers and engineers have at least some exposure to it, and as a result can start using it right away. There are thousands of 3rd party tools that support PostgresSQL that you can just plug in.

When writing plugins, strive for similar simplicity and consistency:

- Follow the Steampipe standards.
- Don't re-invent the wheel - use the names, terms, and values that users are already familiar with. We typically align our table and column names with the equivalent Terraform resource if one is available, and with the API naming if not.

It should be fast (but responsible)

The magic of Steampipe is that it feels like a database, yet it doesn't store any data. We put in quite a lot of effort to make it feel fast and responsive, minimizing the number of API calls based on the request, using multi-threading to parallelize requests, and streaming results.

While much of this work is handled by Steampipe itself, you should endeavor to keep things tight in your plugins as well:

- Don't make extraneous API calls.
- Make intelligent use of caching.
- Back off intelligently if you get throttled by your API.
- Attempt to design tables and columns such that you do not overwhelm the service or API that you are connecting to.

It should be clever and flexible

As we first started building Steampipe, we realized we were on to something because every time someone implemented a new table, someone else came up with new ideas for how to use it. We have added features like autocomplete and `.inspect` to make it easy to discover things, flexible output formats suitable to humans and computers, and utility tables to turn complex json columns in to easy to use tables. We have a big vision for Steampipe, but we sincerely hope that our users -- ***YOU!*** -- do things with Steampipe that we haven't even dreamed of.

When you write your plugin, make hard things easy, and many things possible:

- Normalize complex structures, but make raw json available as well.
- Build something usable and share it as soon as its MVP. Be agile - iterate!
- Design for real use-cases, and imagine possibilities.